TALLER ABP - ENTREGA 1

VeriAI Image

Aplicación móvil para la estimación probabilística de imágenes generadas mediante inteligencia artificial

Curso	Diseño de Aplicaciones Móviles
Metodología académica	Aprender haciendo
Metodología de desarrollo	Lean Mobile (MVP + Iteración Rápida)
Modalidad	Individual
Estudiante	Danilo Alejandro Quintero Gallego
Repositorio	github.com/Daquinterog-UCN/VeriAI-Image
Fecha	14 de octubre de 2026

Propósito: presentar la definición metodológica, el problema, los objetivos SMART, la arquitectura, el alcance, los costes, la seguridad, la estructura del repositorio y el plan de implementación del MVP.

Contenido

1 1. Identificación del proyecto
2 2. Marco metodológico
3 3. Planteamiento del problema
4 4. Pregunta problema y justificación
5 5. Objetivos SMART
6 6. Alcance y requisitos del MVP
7 7. Arquitectura tecnológica
8 8. Diseño UX/UI propuesto
9 9. Plan de implementación y pruebas
10 10. Seguridad, privacidad y uso responsable
11 11. Costes estimados
12 12. Estructura de GitHub y README.md
13 13. Resultados esperados y criterios de aceptación
14 14. Conclusiones y referencias

1. Identificación del proyecto

Título formal	VeriAI Image: Aplicación móvil para la estimación probabilística de imágenes generadas me
Integrante	Danilo Alejandro Quintero Gallego
Metodología	Lean Mobile (MVP + Iteración Rápida)
Enfoque tecnológico	Multiplataforma / Cross-Platform con Flutter
Backend propuesto	Python + FastAPI + API REST
Repositorio público	https://github.com/Daquinterog-UCN/VeriAI-Image

2. Marco metodológico

El proyecto VeriAI Image se desarrollará bajo la metodología Lean Mobile, orientada a construir un Producto Mínimo Viable (MVP) y mejorarlo mediante ciclos cortos de diseño, implementación, prueba y aprendizaje. Esta metodología permite concentrar el esfuerzo inicial en las funciones indispensables: seleccionar o capturar una imagen, enviarla a un servicio de análisis y mostrar una estimación probabilística comprensible.

La primera iteración se enfocará en la investigación del usuario y el diseño UX/UI. La segunda desarrollará el cliente móvil y la integración con la API. La tercera verificará la solución en dispositivos o emuladores mediante pruebas funcionales, de usabilidad y de rendimiento. Cada ciclo producirá evidencias que permitirán corregir problemas antes de ampliar el alcance.

El enfoque metodológico también incorpora una comunicación responsable del resultado. VeriAI Image no afirmará que una imagen es auténtica o sintética con certeza absoluta. El sistema mostrará una estimación, un nivel de confianza y una advertencia sobre falsos positivos, falsos negativos y pérdida de metadatos.

3. Planteamiento del problema

El crecimiento de las herramientas de inteligencia artificial generativa ha facilitado la producción de imágenes sintéticas con un nivel de realismo cada vez mayor. Para una persona usuaria, la inspección visual puede ser insuficiente para distinguir una fotografía convencional de una imagen generada mediante IA. Esta dificultad se incrementa cuando el archivo ha sido recortado, recomprimido, editado o convertido en una captura de pantalla.

La ausencia de una herramienta accesible desde el teléfono limita la posibilidad de realizar una primera revisión técnica del contenido. Por ello se propone una aplicación móvil que integre varias señales: validación del archivo, revisión de metadatos disponibles y consulta de un modelo de clasificación previamente entrenado. La respuesta se presentará de forma sencilla, reconociendo siempre que constituye una estimación y no una prueba definitiva.

4. Pregunta problema

¿De qué manera la implementación de una aplicación móvil integrada con un modelo de análisis de imágenes puede facilitar la estimación de la probabilidad de que una imagen haya sido generada mediante inteligencia artificial?

Justificación

La propuesta es relevante porque combina diseño móvil, experiencia de usuario, consumo de servicios web, procesamiento de archivos, arquitectura en la nube e integración con inteligencia artificial. Además, permite abordar un problema actual desde una perspectiva educativa y responsable,

mostrando resultados interpretables y sus limitaciones.

5. Objetivos SMART

Objetivo general

Desarrollar durante el periodo académico un Producto Mínimo Viable de VeriAI Image, una aplicación móvil multiplataforma que permita seleccionar o capturar imágenes, enviarlas mediante una API REST a un servicio de análisis y presentar una estimación probabilística, una categoría de resultado y un nivel de confianza, verificando su funcionamiento mediante pruebas de carga, respuesta, manejo de errores y usabilidad.

1. Investigación y UX/UI

Identificar las necesidades principales de las personas usuarias y diseñar los wireframes y el prototipo de las pantallas de inicio, selección o captura, vista previa, procesamiento, resultado e información.

2. Desarrollo e integración

Implementar el MVP en Flutter e integrarlo con una API REST desarrollada con FastAPI, encargada de validar, preparar y analizar la imagen mediante un modelo preentrenado.

3. Pruebas y medición

Realizar pruebas funcionales, de usabilidad y rendimiento con distintos tipos de imágenes, verificando carga, comunicación, tiempos de respuesta, presentación de resultados y manejo de errores.

6. Alcance y requisitos del MVP

Funciones incluidas

- Seleccionar imágenes desde la galería.
- Capturar una imagen mediante la cámara.
- Previsualizar y confirmar la imagen antes del envío.
- Validar formato y tamaño del archivo.
- Enviar la imagen al backend mediante HTTPS.
- Obtener una estimación probabilística del modelo.
- Mostrar la categoría: “Probablemente generada mediante IA”, “Probablemente capturada por una cámara” o “Resultado no concluyente”.
- Mostrar nivel de confianza y advertencia de uso.
- Gestionar errores de red, formato, tamaño y respuesta del servidor.
- Permitir realizar un nuevo análisis.

Fuera del alcance inicial

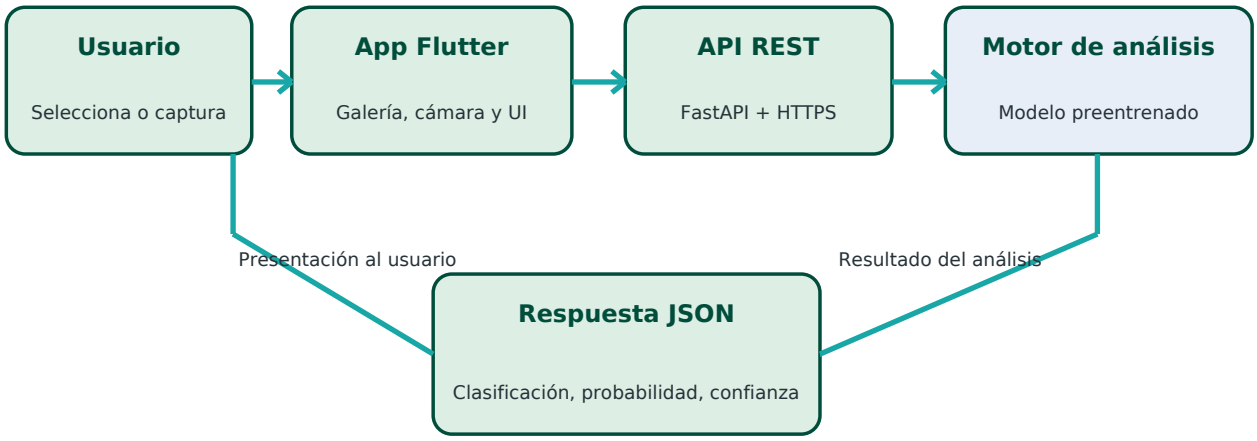
No se entrenará un modelo propio desde cero, no se garantizará una detección infalible, no se realizará reconocimiento de identidad y no se almacenarán permanentemente las imágenes. El MVP priorizará la integración móvil-nube y la representación responsable del resultado.

Requisitos no funcionales

Usabilidad	Flujo breve, mensajes claros y resultado interpretable.
Rendimiento	Indicador de progreso y registro del tiempo de respuesta durante las pruebas.
Compatibilidad	Ejecución inicial en Android mediante Flutter; ampliable a iOS.
Seguridad	HTTPS, validación del archivo y ausencia de secretos en el repositorio.
Privacidad	Procesamiento temporal y eliminación del archivo tras el análisis.
Mantenibilidad	Separación entre cliente móvil, backend y documentación.

7. Arquitectura tecnológica

La solución utilizará una arquitectura cliente-servidor. La aplicación Flutter gestionará la interacción con la persona usuaria. La API FastAPI recibirá el archivo, validará las condiciones de entrada, preparará la imagen y consultará el modelo. La respuesta se enviará en formato JSON con la categoría, la probabilidad, el nivel de confianza y un mensaje explicativo.



Componente	Responsabilidad
App Flutter	Cámara, galería, navegación, validaciones básicas y presentación del resultado.
API FastAPI	Recepción segura, validación, preparación del archivo y orquestación del análisis.
Modelo preentrenado	Generar una puntuación probabilística para las categorías definidas.
Capa de interpretación	Convertir la puntuación en categoría, confianza, explicación y advertencia.
Repositorio GitHub	Código, documentación, presentación y evidencias del proyecto.

8. Diseño UX/UI propuesto

Pantalla	Contenido principal
Inicio	Nombre del proyecto, explicación breve, seleccionar imagen y tomar fotografía.
Vista previa	Imagen elegida, tamaño, formato, botón analizar y opción cambiar.
Procesamiento	Indicador de progreso y mensajes de etapa.
Resultado	Categoría, probabilidad estimada, nivel de confianza, señales y advertencia.
Información	Alcance, privacidad, limitaciones y uso responsable.

Ejemplo de presentación del resultado

Resultado: Probablemente generada mediante IA
Probabilidad estimada: 82 %
Nivel de confianza: Medio

El resultado es orientativo y puede contener falsos positivos o falsos negativos.

9. Plan de implementación y pruebas

Fase	Producto verificable
1. Definición	Problema, pregunta, objetivos, alcance y repositorio.
2. Diseño	Wireframes, prototipo y flujo de navegación.
3. Cliente móvil	Pantallas Flutter, cámara, galería y validaciones.
4. Backend	API FastAPI, endpoint de análisis y control de errores.
5. Integración	Comunicación segura y presentación de la respuesta.
6. Pruebas	Casos funcionales, rendimiento, usabilidad y evidencias.
7. Entrega	README final, código, presentación y enlace público.

Casos básicos de prueba

Caso	Resultado esperado
Imagen JPEG válida	El sistema procesa y muestra una estimación.
Archivo no compatible	La aplicación informa el formato permitido.
Archivo demasiado grande	La carga se bloquea y se muestra una explicación.
Fallo de conexión	Se presenta un mensaje y se permite reintentar.
Respuesta no concluyente	Se evita forzar una clasificación binaria.
Captura de pantalla	Se informa que puede existir menor confianza por pérdida de metadatos.

10. Seguridad, privacidad y uso responsable

La comunicación entre la aplicación y el backend se realizará mediante HTTPS. El backend validará tipo, extensión y tamaño del archivo antes de iniciar el procesamiento. Las imágenes se conservarán únicamente durante el tiempo indispensable para el análisis y se eliminarán después de generar la respuesta. Las claves, tokens y credenciales no se incluirán en el repositorio público; se gestionarán mediante variables de entorno.

La interfaz avisará que el resultado es probabilístico. La aplicación no deberá utilizarse como prueba definitiva, ni como fundamento único para acusaciones, sanciones o decisiones de alto impacto. Los análisis deberán interpretarse junto con otras evidencias y con revisión humana.

11. Costes estimados

Concepto	Proveedor	Frecuencia	Coste estimado
Google Play Console	Google	Pago único	25 USD
Apple Developer Program	Apple	Suscripción anual	99 USD
Backend / base de datos	Google Cloud / servicio equivalente	Estimación mensual	15 USD
GitHub público	GitHub	Plan gratuito	0 USD
Herramientas de desarrollo	Flutter, Dart, Python	Software libre	0 USD
Total estimado inicial			**139 USD**

Para la fase académica, el prototipo puede ejecutarse con emuladores, dispositivos de prueba y planes gratuitos. Las cuentas de tiendas solo serán necesarias si se decide publicar la aplicación.

12. Estructura de GitHub y README.md

El repositorio público deberá contener el código completo, el README estructurado, la presentación y las evidencias. La estructura propuesta es:

```
VeriAI-Image/
├── README.md
├── mobile/     # aplicación Flutter
├── backend/   # API FastAPI y modelo
├── docs/
│   ├── Actividad_1_VerIAI_Image_Completa.pdf
│   ├── arquitectura.png
│   └── evidencias/
├── .gitignore
├── LICENSE
└── .env.example
```

Contenido mínimo del README.md

- Descripción del proyecto y situación problema.
- Pregunta problema, objetivo general y objetivos específicos.
- Metodología y alcance del MVP.
- Tecnologías y arquitectura.
- Estructura del repositorio.
- Requisitos previos e instrucciones de instalación.
- Configuración del backend y variables de entorno.

- Ejecución de Flutter y FastAPI.
- Pruebas, evidencias y resultados.
- Seguridad, privacidad, limitaciones y autoría.

Pasos para completar el repositorio

1. Crear o mantener el repositorio público VeriAI-Image.
2. Agregar README.md en la raíz.
3. Crear las carpetas mobile, backend y docs.
4. Subir este PDF a docs.
5. Agregar el código fuente cuando se implemente.
6. Incluir .gitignore para excluir archivos temporales, compilaciones y secretos.
7. Agregar .env.example con nombres de variables, sin valores reales.
8. Registrar cambios mediante commits descriptivos.
9. Verificar el enlace en una ventana privada antes de entregarlo en Canvas.

13. Resultados esperados y criterios de aceptación

Criterio	Condición de aceptación
Carga de imagen	La persona usuaria puede seleccionar o capturar una imagen válida.
Procesamiento	La API recibe la imagen y devuelve una respuesta estructurada.
Interpretación	La aplicación muestra categoría, probabilidad y nivel de confianza.
Manejo de errores	La interfaz informa fallos de archivo, red o servidor.
Privacidad	La solución no conserva permanentemente la imagen analizada.
Documentación	El repositorio contiene código, README, presentación y evidencias.
Trazabilidad	Los commits y carpetas permiten identificar los componentes del proyecto.

Indicadores de prueba propuestos

Durante la implementación se documentarán: cantidad de casos ejecutados, casos satisfactorios y fallidos, tiempo observado de respuesta por caso, errores encontrados, correcciones realizadas y observaciones de usabilidad. Los valores reales se incorporarán cuando exista un prototipo funcional; no se presentan métricas simuladas como resultados.

14. Conclusiones

VeriAI Image constituye una propuesta viable para aplicar conocimientos de diseño móvil, integración de APIs, arquitectura en la nube e inteligencia artificial. El alcance recomendado evita entrenar inicialmente un modelo propio y concentra el trabajo en un MVP demostrable. La principal consideración metodológica es comunicar el resultado como estimación probabilística y no como certificación de autenticidad.

La entrega quedará completa cuando el repositorio incluya el cliente Flutter, el backend FastAPI, las instrucciones de despliegue, el PDF del proyecto y evidencias verificables de ejecución. Hasta entonces, este documento presenta la definición integral y el plan de construcción de la solución.

Referencias técnicas

- GitHub Docs. About the repository README file. https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes
- GitHub Docs. Adding a file to a repository. https://docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository
- Microsoft Learn. Provenance detection overview. https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/provenance-detection